# KENFINLY — Financial Analytics Screen

BUSINESS SPECIFICATION & SYSTEM ARCHITECTURE

Business specification and system architecture document for the **Core Analytics** module of KenFinly — an automated personal and small-business financial health tracking tool that helps users **"see" cash flow in just 3 seconds**. This module ingests data from the Transaction stream, processes it through the Aggregation Layer, and displays it in the PWA/Desktop App.

# Overview & Business Objectives

### 🎯 Objective

Automate financial health tracking and reduce churn rate through data-driven engagement.

### 💡 Value

Helps users "see" cash flow in 3 seconds. Automatically calculates spending proportions to cut unnecessary expenses.

### 🔧 Problem Solved

Replaces manual bookkeeping; addresses unclear reasons for budget shortfalls.

### 📊 KPIs Tracked

DAU/MAU on the Analytics feature. Time on Page — time spent interacting with charts and filters.

# Overall Interface Architecture

The system is designed with a **Mobile-First** approach. All UI on the Mobile PWA becomes independent Mobile Components, then is embedded into the Desktop interface as a **Grid** — optimizing space and reusing 100% of frontend logic.

1

### Header & Filter Zone

The navigation area and dynamic time filter (Date Range Filter) — located at the very top.

2

### Overview Area

Summary card showing financial progress comparisons using the **MoM** (Month-over-Month) model.

3

### Visual Chart Area

Customizable bar chart or line chart based on the selected time period.

4

### Data Table Area

Structured expense list (Top Category) with percentage allocation breakdown.

# Time Filter & Header Component

### Component 1: Local Header

**Type:** Organism / Fixed Global Header

Displays the text **"Financial Analysis"** (font-weight: bold). The right side integrates a Shortcut/Notification icon.

**Loading state:** Show a skeleton for the text area.

### Component 2: Date Range Filter

**Type:** Molecule / Button Group Filter — changes the query range for the entire widget below.

| Filter | Data Range |
|---|---|
| Today | 00:00 – 23:59 of the current day |
| Last 7 Days | 7 days counted backward from today |
| This Month | From the 1st to the end of the current month |
| Last Month | From the 1st to the end of the previous month |
| This Year | 01/01 – 31/12 of the current year |
| Custom | [Start Date – End Date] selected manually |

**Edge Case:** If the selected time range is > 3 years, the system automatically groups the X-axis by Quarter/Year to avoid UI overlap.

# Chart Component & Summary Card

### Component 3: Summary Card (MoM)

Compare financial performance between **This Month vs Last Month**. Includes 2 radial progress circles — similar to the /Home page.

| Circle 1 | Circle 2 |
| --- | --- |
| This Month | Last Month |

### Component 4: Bar/Combo Chart

Visualize the balance of **Income – Expenses – Accumulated**.

- **X-axis:** Time markers (T01 → T06...)
- **Y-axis:** Abbreviated monetary values (40.00M, 80.00M...)
- **Legend:** Income (Blue), Expenses (Red), Accumulated (Yellow/Green)
- **Tooltip:** Tap any bar to view detailed Income/Expenses/Accumulated data
- **Loading:** Opacity 0.5 + circular spinner at the center of the chart

# Top Category Data & Metrics Table

### 🍔 Category

Category name + icon (🍔 Food & Dining, 🚗 Transportation, 🏠 Rent).

### 💰 Total Amount

Negative amounts in red (Example: **-4.5Mđ**). Rounded to 1 decimal place.

### 🔢 Transaction Count

Standard format: **[Count] + " TX"** (Example: 12 TX, 1 TX).

### 📊 % Spend

Category share of total spending. **Total must equal 100%** (any rounding difference is added to or subtracted from the largest category).

**% Spend formula:** (Category total spend ÷ Total overall spending) × 100, rounded with Math.round(). If the total is 99% or 101%, the difference is adjusted in the category with the largest share.

# Business Rules & API Requirements

### Business Rules

**BR-001 — Currency synchronization:** All currency units follow the account's local settings (Default: ₫ - VND).

**BR-002 — Future transactions:** When selecting a date that has not yet occurred, the system displays **"Projected budget"** if the user has a preconfigured setting.

### API Endpoint

`GET /api/v1/analytics/summary`

```
{
  "range_type": "7_DAYS",
  "start_date": "2026-06-22",
  "end_date": "2026-06-28"
}
```

Response returns: **overview** (MoM), **charts** (income/expense/savings), **top_categories** (id, name, icon, total_spend, transaction_count, percentage_spend).

→ **FR-ANL-001: Filter data by time range**

The user selects a filter → the entire card, charts, and table update **in real time without reloading the page** (No-reload SPA).

→ **FR-ANL-002: Standardize the GD count string**

The frontend receives `transaction_count` from the API → outputs a string formatted as **"12 GD"**. It must never produce concatenation errors like "128GD1GD".

# Technical Architecture: Database & Backend

### Database — Pre-aggregated Table

To optimize query performance (avoiding SUM across millions of raw records), the system uses a pre-aggregated table:

```
CREATE TABLE summary_category_daily
(
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  category_id INT NOT NULL,
  log_date DATE NOT NULL,
  total_spend NUMERIC(15,2) DEFAULT 0.00,
  transaction_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_user_date_analytics
  ON summary_category_daily(user_id, log_date);
```

### Backend Strategy

#### Redis Cache Layer

When a request arrives, the Service Layer checks the cache key `analytics:user:{id}:range:{type}`. If it exists → return immediately **(< 50ms)**.

#### Cron Job Syncing

The worker automatically runs at **00:01 every day** to precompute the previous day's data and load it into `summary_category_daily`.

# Frontend Architecture & QA Testing

## Frontend Requirements

### Atomic Design

Separate **WidgetChart**, **WidgetTable**, **FilterGroup** into independent components.

### Axios Client

A unified client for both PWA Build and Desktop Build with `baseURL: ${API_URL}`.

### Redux Toolkit

Manage global filter state (`activeFilter`). When it changes → trigger simultaneous fetching for all child widgets.

## QA Test Scenarios

| | |
|---|---|
| **TC-ANL-01** | **Happy Path:** Select the 7-day filter → The chart displays exactly 7 points, and the table updates with the correct amounts. |
| **TC-ANL-02** | **Validation:** Check the transaction count column → Standard format "12 transactions", with no concatenation errors. |
| **TC-ANL-03** | **Edge Case:** Select a range with no transactions → Show an Empty State, no crash/NaN%. |
| **TC-ANL-04** | **Responsive:** Mobile PWA → Desktop: Grid rearranges neatly, with no broken layout/overlapping blocks. |

⚠ **Technical edge cases:** Network loss → read data from LocalStorage/IndexedDB + Toast "You are viewing offline data". API Timeout >10s → display a "Reload data" button.

# Non-Functional Requirements & Implementation Plan

| <1.5s | <50ms | 100% |
| --- | --- | --- |
| **FCP Target** | **Cache Response** | **% Spend Total** |
| First Contentful Paint on 3G/4G networks must be under 1.5 seconds. | Time to return data from Redis Cache on a cache hit. | The total percentage of spending across all categories must always equal 100%. |

### 🔒 Security

All APIs require a **Bearer Token (JWT)** in the Header. The system encrypts users' financial data when stored.

### 💡 UI/UX Recommendations

Add a toggle button to hide/show amounts (***) when used in public places. On screens <360px, automatically switch the Bar Chart to a **Scrollable Line Chart**.

## Agile Development Plan

01

### Phase 1 — MVP (Highest Priority)

Complete the mobile PWA UI, implement the **7-day** filter API, and fix the Số GD string formatting issue.

02

### Phase 2

Deploy a **Caching Layer** to optimize performance and package Mobile Components for Desktop.